E-Commerce Analysis

Required Library

In [1]:
```
import urllib.parse
from sqlalchemy import create_engine
import time
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')
```

Load DataSet

In [2]:
```
df =
pd.read_csv(r"C:\Users\dell\Downloads\messy_ecommerce_sales_data.csv")
```

In [3]:
```
df.head()
```

Out[3]:

	ID	Customer_Name	Order_ID	Order_Date	Product	Cat
0	100	Customer_100	ORD-41285	2024-11-22	Blender	Hom
1	101	Customer_101	ORD-35783	2025-07-05	Smartphone	Elect
2	102	Customer_102	ORD-84355	2024-12-23	Tennis Racket	Spor
3	103	Customer_103	ORD-57811	2025-03-19	Science	Bool
4	104	Customer_104	ORD-93614	2025-10-20	Biography	Bool

In [4]:
```
# Copy first orginal Data
df_Clean = df.copy()
```

Data Preaparing & Cleaning Process

In [5]:
```
# Basic Informations
rows = df_Clean.shape[0]
columns = df_Clean.shape[1]
print(f'The Dataset have total number of {rows} rows & {columns}
columns')
```

```
The Dataset have total number of 103 rows & 11 columns
```

In [6]:
```
# Information of data set
df_Clean.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 103 entries, 0 to 102
Data columns (total 11 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   ID              103 non-null    int64
 1    Customer_Name  103 non-null    object
 2   Order_ID        103 non-null    object
 3   Order_Date      103 non-null    object
 4    Product        103 non-null    object
 5   Category        95 non-null     object
 6   Quantity        98 non-null     object
 7   Price           98 non-null     object
 8   Payment_Method  103 non-null    object
 9   Status          103 non-null    object
 10  Total           89 non-null     float64
dtypes: float64(1), int64(1), object(9)
memory usage: 9.0+ KB
```

In [7]:
```
# Reomove space on every column
df_Clean.columns = df_Clean.columns.str.strip()
```

In [8]:
```
for col in df_Clean.columns:
    print(df_Clean[col].value_counts())
    print("--"*60)
```

```
ID
146    2
175    2
142    2
100    1
104    1
      ..
195    1
196    1
197    1
198    1
199    1
Name: count, Length: 100, dtype: int64
------------------------------------------------------------------
-------------------------------------------------------
Customer_Name
Customer_146    2
Customer_175    2
Customer_142    2
Customer_100    1
Customer_104    1
               ..
Customer_195    1
Customer_196    1
Customer_197    1
Customer_198    1
Customer_199    1
Name: count, Length: 100, dtype: int64
------------------------------------------------------------------
-------------------------------------------------------
Order_ID
ORD-32755    2
ORD-56651    2
ORD-69018    2
ORD-41285    1
ORD-93614    1
            ..
ORD-82876    1
ORD-78384    1
ORD-79139    1
ORD-14608    1
ORD-82922    1
Name: count, Length: 100, dtype: int64
------------------------------------------------------------------
-------------------------------------------------------
Order_Date
2025-02-09    4
2025-10-30    3
2025-10-11    2
2025-05-31    2
2025-01-03    2
             ..
```

```
2025-02-10    1
2025-06-19    1
2025-06-23    1
2025-07-27    1
2025-01-22    1
Name: count, Length: 86, dtype: int64
-----------------------------------------------------------------
-------------------------------------------------------
Product
Blender          8
Shoes            8
Basketball       8
Tennis Racket    7
Comics           7
Lamp             7
Science          6
Biography        5
Smartphone       5
Fiction          5
Microwave        5
Yoga Mat         5
Vacuum           5
Laptop           4
T-shirt          3
Headphones       3
Smartwatch       3
Jeans            3
Jacket           3
Football         2
shoes            1
Name: count, dtype: int64
-----------------------------------------------------------------
-------------------------------------------------------
Category
Books          22
Home           20
Sports         16
Clothing       15
Electronics    11
electronic      4
ELECTRONICS     3
electronics     3
sports          1
Name: count, dtype: int64
-----------------------------------------------------------------
-------------------------------------------------------
Quantity
5     25
2     21
1     20
4     16
3     13
-2     1
-5     1
4a     1
Name: count, dtype: int64
-----------------------------------------------------------------
```

```
-----------------------------------------------------
Price
abd       2
705.42    2
645.26    2
111.36    2
389.05    1
         ..
944.54    1
817.46    1
160.16    1
497.01    1
372.28    1
Name: count, Length: 94, dtype: int64
-------------------------------------------------------------------
-----------------------------------------------------
Payment_Method
Cash on Delivery    33
PayPal              24
Bank Transfer       23
Credit Card         23
Name: count, dtype: int64
-------------------------------------------------------------------
-----------------------------------------------------
Status
Processing    27
Returned      27
Shipped       23
Cancelled     15
Delivered     11
Name: count, dtype: int64
-------------------------------------------------------------------
-----------------------------------------------------
Total
1410.840    2
114.000     1
1169.600    1
389.050     1
366.180     1
           ..
160.160     1
994.020     1
1861.400    1
77.952      1
3226.300    1
Name: count, Length: 88, dtype: int64
-------------------------------------------------------------------
-----------------------------------------------------
```

In [9]:
```
for col in df_Clean.columns:
    print(df_Clean[col].unique())
    print("--"*60)
```

```
[100 101 102 103 104 105 106 107 108 109 110 111 112 113 114 115
116 117
 118 119 120 121 122 123 124 125 126 127 128 129 130 131 132 133
134 135
 136 137 138 139 140 141 142 143 144 145 146 147 148 149 150 151
152 153
 154 155 156 157 158 159 160 161 162 163 164 165 166 167 168 169
170 171
 172 173 174 175 176 177 178 179 180 181 182 183 184 185 186 187
188 189
 190 191 192 193 194 195 196 197 198 199]
------------------------------------------------------------------
-------------------------------------------------------
['Customer_100' 'Customer_101' 'Customer_102' 'Customer_103'
 'Customer_104' 'Customer_105' 'Customer_106' 'Customer_107'
 'Customer_108' 'Customer_109' 'Customer_110' 'Customer_111'
 'Customer_112' 'Customer_113' 'Customer_114' 'Customer_115'
 'Customer_116' 'Customer_117' 'Customer_118' 'Customer_119'
 'Customer_120' 'Customer_121' 'Customer_122' 'Customer_123'
 'Customer_124' 'Customer_125' 'Customer_126' 'Customer_127'
 'Customer_128' 'Customer_129' 'Customer_130' 'Customer_131'
 'Customer_132' 'Customer_133' 'Customer_134' 'Customer_135'
 'Customer_136' 'Customer_137' 'Customer_138' 'Customer_139'
 'Customer_140' 'Customer_141' 'Customer_142' 'Customer_143'
 'Customer_144' 'Customer_145' 'Customer_146' 'Customer_147'
 'Customer_148' 'Customer_149' 'Customer_150' 'Customer_151'
 'Customer_152' 'Customer_153' 'Customer_154' 'Customer_155'
 'Customer_156' 'Customer_157' 'Customer_158' 'Customer_159'
 'Customer_160' 'Customer_161' 'Customer_162' 'Customer_163'
 'Customer_164' 'Customer_165' 'Customer_166' 'Customer_167'
 'Customer_168' 'Customer_169' 'Customer_170' 'Customer_171'
 'Customer_172' 'Customer_173' 'Customer_174' 'Customer_175'
 'Customer_176' 'Customer_177' 'Customer_178' 'Customer_179'
 'Customer_180' 'Customer_181' 'Customer_182' 'Customer_183'
 'Customer_184' 'Customer_185' 'Customer_186' 'Customer_187'
 'Customer_188' 'Customer_189' 'Customer_190' 'Customer_191'
 'Customer_192' 'Customer_193' 'Customer_194' 'Customer_195'
 'Customer_196' 'Customer_197' 'Customer_198' 'Customer_199']
------------------------------------------------------------------
-------------------------------------------------------
['ORD-41285' 'ORD-35783' 'ORD-84355' 'ORD-57811' 'ORD-93614' 'ORD-
22442'
 'ORD-25885' 'ORD-89122' 'ORD-64400' 'ORD-18512' 'ORD-61020' 'ORD-
72349'
 'ORD-80867' 'ORD-23994' 'ORD-77417' 'ORD-72778' 'ORD-63660' 'ORD-
72751'
 'ORD-96061' 'ORD-76016' 'ORD-14659' 'ORD-80067' 'ORD-21443' 'ORD-
11314'
 'ORD-46136' 'ORD-23202' 'ORD-11261' 'ORD-65482' 'ORD-12270' 'ORD-
48863'
 'ORD-34007' 'ORD-91396' 'ORD-40186' 'ORD-68182' 'ORD-16585' 'ORD-
33656'
```

```
 'ORD-20985' 'ORD-91254' 'ORD-34319' 'ORD-38418' 'ORD-67096' 'ORD-
45059'
 'ORD-69018' 'ORD-14329' 'ORD-89452' 'ORD-75314' 'ORD-32755' 'ORD-
80707'
 'ORD-38423' 'ORD-95911' 'ORD-76902' 'ORD-56552' 'ORD-72869' 'ORD-
29721'
 'ORD-95990' 'ORD-20182' 'ORD-34679' 'ORD-34407' 'ORD-35817' 'ORD-
98162'
 'ORD-96037' 'ORD-27653' 'ORD-27743' 'ORD-24378' 'ORD-23010' 'ORD-
55695'
 'ORD-97756' 'ORD-30329' 'ORD-53430' 'ORD-75539' 'ORD-21530' 'ORD-
99790'
 'ORD-48008' 'ORD-42171' 'ORD-16766' 'ORD-56651' 'ORD-95794' 'ORD-
64136'
 'ORD-89906' 'ORD-99328' 'ORD-86629' 'ORD-54481' 'ORD-79672' 'ORD-
20916'
 'ORD-67799' 'ORD-57123' 'ORD-90569' 'ORD-41292' 'ORD-58590' 'ORD-
30627'
 'ORD-38469' 'ORD-41810' 'ORD-35144' 'ORD-42475' 'ORD-69764' 'ORD-
82876'
 'ORD-78384' 'ORD-79139' 'ORD-14608' 'ORD-82922']
--------------------------------------------------------------------
-------------------------------------------------------
['2024-11-22' '2025-07-05' '2024-12-23' '2025-03-19' '2025-10-20'
 '2024-11-20' '2025-02-02' '2025-01-03' '2025-10-23' '2025-05-03'
 '2025-09-26' '2025-10-22' '2025-03-02' '2025-07-24' 'Jan 5 2023'
 '2025-01-10' '2025-10-30' '2025-02-12' '2025-05-31' '2024-12-19'
 '2025-06-13' '2025-10-11' '2025-05-16' '2025-10-06' '2025-02-09'
 '2025-05-18' '2025-05-12' '2024-11-10' '2025-06-22' '2025-08-15'
 '2025-06-11' '2025-04-02' '2024-12-05' '2025-10-04' '2025-06-12'
 '2025-02-23' '2025-08-09' '2025-02-26' '2025-02-13' '2025-07-11'
 '2025-01-27' '05/01/2023' '2025-07-09' '2025-05-01' '2025-03-13'
 '2025-06-29' '2024-11-27' '2025-07-01' '2025-09-22' '2025-07-16'
 '2025-02-21' '2024-11-28' '2025-11-06' '2025-04-05' '2025-07-28'
 '2024-12-15' '2025-05-19' '2023-01-05' '2024-11-08' '2025-07-03'
 '2025-08-07' '2025-11-04' '2025-08-12' '2025-10-03' '2024-12-06'
 '2024-11-18' '2025-09-20' '2025-02-24' '2025-07-22' '2025-09-18'
 '2025-09-03' '2025-03-26' '2024-12-29' '2025-08-03' '2025-03-10'
 '2025-02-15' '2025-08-14' '2024-11-19' '2025-07-18' 'abc' '2025-
06-06'
 '2025-02-10' '2025-06-19' '2025-06-23' '2025-07-27' '2025-01-22']
--------------------------------------------------------------------
-------------------------------------------------------
['Blender' 'Smartphone' 'Tennis Racket' 'Science' 'Biography'
'Basketball'
 'Comics' 'Shoes' 'Fiction' 'Microwave' 'Yoga Mat' 'Vacuum' 'Lamp'
 'T-shirt' 'Smartwatch' 'Headphones' 'Football' 'Laptop' 'Jacket'
'shoes'
 'Jeans']
--------------------------------------------------------------------
-------------------------------------------------------
['Home' 'Electronics' 'Sports' 'Books' 'sports' 'Clothing'
'electronic'
 'ELECTRONICS' 'electronics' nan]
--------------------------------------------------------------------
-------------------------------------------------------
['3' '2' '1' '5' nan '4' '-2' '-5' '4a']
```

```
--------------------------------------------------------------------
-------------------------------------------------------
['38' 'abd' '389.05' '233.92' '552.51' '122.06' '978.63' '587.68'
'600.29'
 '168.34' 'four hundred' '865.77' '752.13' '411.81' '413.13'
'796.84' nan
 '10000' '247.82' '438.19' '300$' '523.01' '556.93' '615.91'
'765.69'
 '836.8' '336.02' '898.69' '736.71' '616.63' '266.7' '343.24'
'591.53'
 '446.19' '696.71' '-100' '531.06' '681.38' '588.98' '546.04'
'645.26'
 '826.77' '849.07' '860.99' '705.42' '596.15' '660.98' '896.38'
'206.19'
 '457.16' '233.11' '278.9' '789.64' '40.95' '275.1' '933.58'
'163.16'
 '242.52' '209.32' '155.76' '235.26' '587.64' '443.13' '569.38'
'593.93'
 '141.49' '582.13' '741.63' '688.68' '942.76' '351.89' '187.48'
'111.36'
 '530.52' '69.48' '472.14' '868.67' '418.38' '266.48' '538.35'
'101.19'
 '730.92' '365.34' '882.96' '504.05' '814.68' '920.37' '881.02'
'203.63'
 '522.02' '944.54' '817.46' '160.16' '497.01' '372.28']
--------------------------------------------------------------------
-------------------------------------------------------
['Cash on Delivery' 'PayPal' 'Bank Transfer' 'Credit Card']
--------------------------------------------------------------------
-------------------------------------------------------
['Shipped' 'Processing' 'Delivered' 'Cancelled' 'Returned']
--------------------------------------------------------------------
-------------------------------------------------------
[ 1.140000e+02           nan  3.890500e+02  1.169600e+03
5.525100e+02
  3.661800e+02  2.938400e+03  6.002900e+02  8.417000e+02
1.731540e+03
  3.008520e+03  4.118100e+02  8.262600e+02  1.115576e+03
-2.000000e+05
  1.239100e+03  1.752760e+03  2.092040e+03  1.113860e+03
1.847730e+03
  3.062760e+03  3.347200e+03  1.680100e+03  2.696070e+03
2.946840e+03
  3.083150e+03  2.667000e+02  1.716200e+03 -2.957650e+03
1.784760e+03
  6.967100e+02 -3.000000e+02  5.310600e+02  1.362760e+03
2.355920e+03
  1.638120e+03  2.258410e+03  4.133850e+03  4.245350e+03
1.721980e+03
  1.410840e+03  2.384600e+03  6.609800e+02  4.481900e+03
2.061900e+02
  6.400240e+02  6.993300e+02  1.115600e+03  3.948200e+03
4.095000e+01
  1.100400e+03  9.335800e+02  4.894800e+02  1.212600e+03
8.372800e+02
  3.115200e+02  7.057800e+02  8.862600e+02  1.138760e+03
2.829800e+02
```

```
  5.821300e+02  3.708150e+03  2.066040e+03  1.885520e+03
1.759450e+03
  7.499200e+02  1.113600e+02  2.652600e+03  6.948000e+01
1.416420e+03
  2.606010e+03  8.367600e+02  2.664800e+02  2.153400e+03
4.047600e+02
  1.023288e+03  3.653400e+02  2.648880e+03  2.520250e+03
8.146800e+02
  9.203700e+02  1.762040e+03  4.722700e+03  4.087300e+03
1.601600e+02
  9.940200e+02  1.861400e+03  7.795200e+01  3.226300e+03]
-------------------------------------------------------------------
-----------------------------------------------------
```

In [10]:

```
# Fixing Categorical Issues
df_Clean['Product'].replace({'shoes':'Shoes'}, inplace=True)
(df_Clean['Category'].replace({'sports': 'Sports',
"electronic": "Electronics", 'ELECTRONICS':
'Electronics', 'electronics':'Electronics'},inplace=True))
```

In [11]:

```
#change data types & fix error
df_Clean['Quantity'] = df_Clean['Quantity'].str.replace("-",
'',regex=False )
df_Clean['Quantity'] = df_Clean['Quantity'].str.extract('(\d+)')
df_Clean['Quantity'] = pd.to_numeric(df_Clean['Quantity'])
```

In [12]:

```
df_Clean['Price'] = df_Clean['Price'].str.extract('(\d+\.?\d*)')
df_Clean['Price'] = pd.to_numeric(df_Clean['Price'], errors= 'coerce')
```

In [13]:

```
# Check Duplicate Values
duplicate = df_Clean[df_Clean.duplicated()].shape[0]
print(f"Total Number of Duplicate Records: {duplicate}")
```

```
Total Number of Duplicate Records: 1
```

In [14]:

```
# remove dupplicate records
df_Clean.drop_duplicates(inplace=True)
duplicate = df_Clean[df_Clean.duplicated()].shape[0]
print(f"Total Number of Duplicate Records: {duplicate}")
```

```
Total Number of Duplicate Records: 0
```

In [15]:
```
# Check Missing & fill Values
Null_Summary = pd.DataFrame({"Number of Null": df_Clean.isnull().sum(),
'Percentages': round((df_Clean.isnull().sum()/len(df_Clean))*100,2)})
Null_Summary
```

Out[15]:

	Number of Null	Percentages
ID	0	0.00
Customer_Name	0	0.00
Order_ID	0	0.00
Order_Date	0	0.00
Product	0	0.00
Category	8	7.84
Quantity	5	4.90
Price	8	7.84
Payment_Method	0	0.00
Status	0	0.00
Total	14	13.73

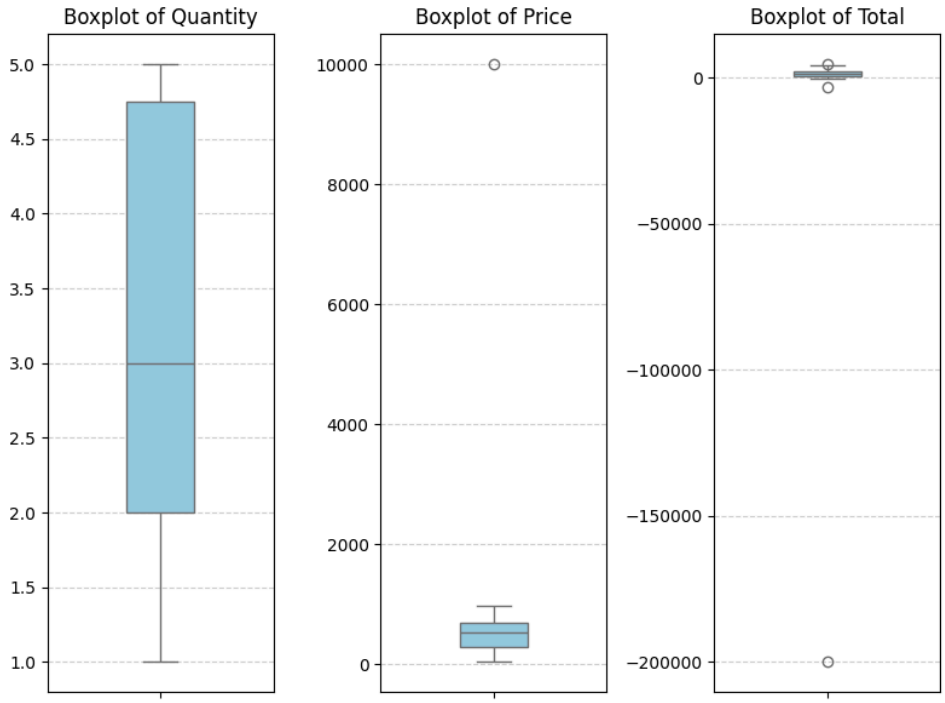
In [16]:
```
Total_Null = round((df_Clean.isnull().sum().sum()/df_Clean.size)*100,2)
print(f'Overall total Null {Total_Null}%')
```

```
Overall total Null 3.12%
```

In [17]:
```
df_Clean['Category'].fillna(df_Clean['Category'].mode()[0],inplace=True)
```

In [18]:
```
# price fill
df_Clean['Price'] = df_Clean['Price'].fillna(df_Clean['Price'].median())
# quantity fill
df_Clean['Quantity'] =
df_Clean['Quantity'].fillna(df_Clean['Quantity'].median())
df_Clean['Quantity'] = df_Clean['Quantity'].astype(int)
# total fill
df_Clean['Total'] = df_Clean['Total'].fillna(
    df_Clean['Price'] * df_Clean['Quantity']
)
```

In [19]:
```
# check outlier & Handling Outlier
cols = ['Quantity','Price', 'Total']
plt.figure(figsize=(8,6))
for i, col in enumerate(cols,1):
    plt.subplot(1,3,i)
    sns.boxplot(y= df_Clean[col], color='skyblue', width=0.3)
    plt.title(f'Boxplot of {col}', fontsize = 12)
    plt.ylabel('')
    plt.grid(axis='y', linestyle = '--', alpha = 0.6)
plt.tight_layout()
plt.show()
```



In [20]:
```
demo = df_Clean[df_Clean['Price'] > 5000]
demo
```

Out[20]:

	ID	Customer_Name	Order_ID	Order_Date	Product	Categ
17	117	Customer_117	ORD-72751	2025-02-12	Blender	Electron

In [21]:
```
df_Clean['Total'] = np.abs(df_Clean['Total'])
```

In [22]:
```
df_Clean['Total'] = df_Clean['Price'] * df_Clean['Quantity']
```

In [23]:
```
(df_Clean['Total'] == df_Clean['Price'] * df_Clean['Quantity']).all()
```

Out[23]: np.True_

In [24]:
```
# Fix date Columns
df_Clean['Order_Date'] = pd.to_datetime(
    df_Clean['Order_Date'],
    errors='coerce',
    dayfirst=True
)
```

In [25]:
```
df_Clean = df_Clean.dropna(subset=['Order_Date'])
```

In [26]:
```
df_Clean.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 99 entries, 0 to 101
Data columns (total 11 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   ID              99 non-null     int64
 1   Customer_Name   99 non-null     object
 2   Order_ID        99 non-null     object
 3   Order_Date      99 non-null     datetime64[ns]
 4   Product         99 non-null     object
 5   Category        99 non-null     object
 6   Quantity        99 non-null     int64
 7   Price           99 non-null     float64
 8   Payment_Method  99 non-null     object
 9   Status          99 non-null     object
 10  Total           99 non-null     float64
dtypes: datetime64[ns](1), float64(2), int64(2), object(6)
memory usage: 9.3+ KB
```

Load Clean DataSet in MySQL

In [27]:

```
username = "root"
raw_password = "ANku7970@!#"
password = urllib.parse.quote_plus(raw_password)

host = "localhost"
port = 3306
database = "Retail_Analysis"

engine = create_engine(f"mysql+pymysql://{username}:{password}@{host}:
{port}/{database}")

try:
   print("Attempting to connect to database...")
   for x in range(1,4):
      print(x)
      time.sleep(1)
   df_Clean.to_sql("df_clean",engine,
   if_exists="replace",index=False)
   print("Connected ✅ Data Loaded Successfully")

except Exception as e:
   print("Not Connected❌")
   print("Reson",e)
```

```
Attempting to connect to database...
1
2
3
Connected ✅ Data Loaded Successfully
```

Exploratory Data Analysis & Visualizations

In [28]:
```
df_Clean.describe()
```

Out[28]:

	ID	Order_Date	Quantity	Price	
count	99.000000	99	99.000000	99.000000	9
mean	149.656566	2025-05-02 07:01:49.090909184	3.090909	608.239596	1
min	100.000000	2023-01-05 00:00:00	1.000000	38.000000	4
25%	125.500000	2025-02-09 00:00:00	2.000000	289.450000	7
50%	150.000000	2025-05-19 00:00:00	3.000000	534.705000	1
75%	174.500000	2025-08-05 00:00:00	5.000000	701.065000	2
max	199.000000	2025-11-06 00:00:00	5.000000	10000.000000	2
std	28.739208	NaN	1.492094	985.996648	2

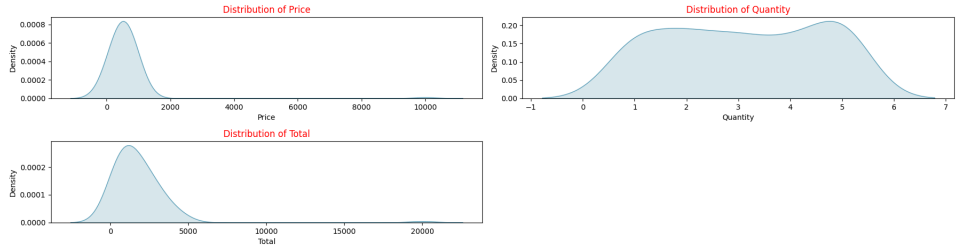
In [29]:
```
df_Clean.describe(include='object').T
```

Out[29]:

	count	unique	top	freq
Customer_Name	99	97	Customer_142	2
Order_ID	99	97	ORD-69018	2
Product	99	20	Shoes	9
Category	99	5	Books	30
Payment_Method	99	4	Cash on Delivery	32
Status	99	5	Processing	26

Distribution of Numberical Columns

In [30]:
```
Num_column = ['Price','Quantity','Total']
plt.figure(figsize=(18,9))
for x, col in enumerate(Num_column):
    plt.subplot(4,2,x+1)
    sns.kdeplot(x = df_Clean[col],fill=True, color='#60a3bc')
    plt.title(f"Distribution of {col}", fontsize = 12,
    color = "red")
plt.tight_layout()
plt.show()
```



Insight

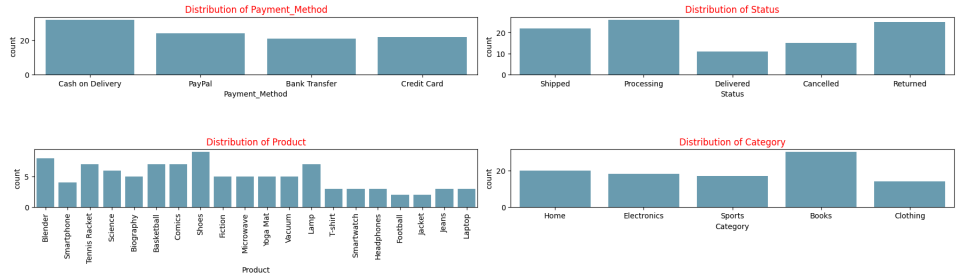
1. Price Distribution Insight
- The distribution is right-skewed (positively skewed)
- Most values are concentrated in the low to mid range (0–1000)
- A few values extend up to 10,000 (high-end products)

2. Quantity Distribution Insight
- Values mostly lie between 1 to 5 units
- Distribution is relatively balanced (not heavily skewed)

3. Total (Revenue) Distribution Insight
- The distribution is right-skewed
- Most transactions are low to medium value
- A few transactions have very high total values

Distribution of Categorical Columns

In [31]:
```
Categorical_column = ['Payment_Method','Status', "Product", "Category"]
plt.figure(figsize=(18,9))
for x, col in enumerate(Categorical_column):
    plt.subplot(4,2,x+1)
    sns.countplot(x=df_Clean[col],color='#60a3bc')
    plt.title(f"Distribution of {col}", fontsize = 12,
    color = "red")
    if col =='Product':
        plt.xticks(rotation = 90)
plt.tight_layout()
plt.show()
```



Insights

Payment Method
- Cash on Delivery is the most preferred payment method, indicating that customers may have lower trust in online payment systems, which can impact digital payment adoption.

Status
- A high number of returned orders suggests potential issues in product quality or delivery, which may negatively affect customer satisfaction.

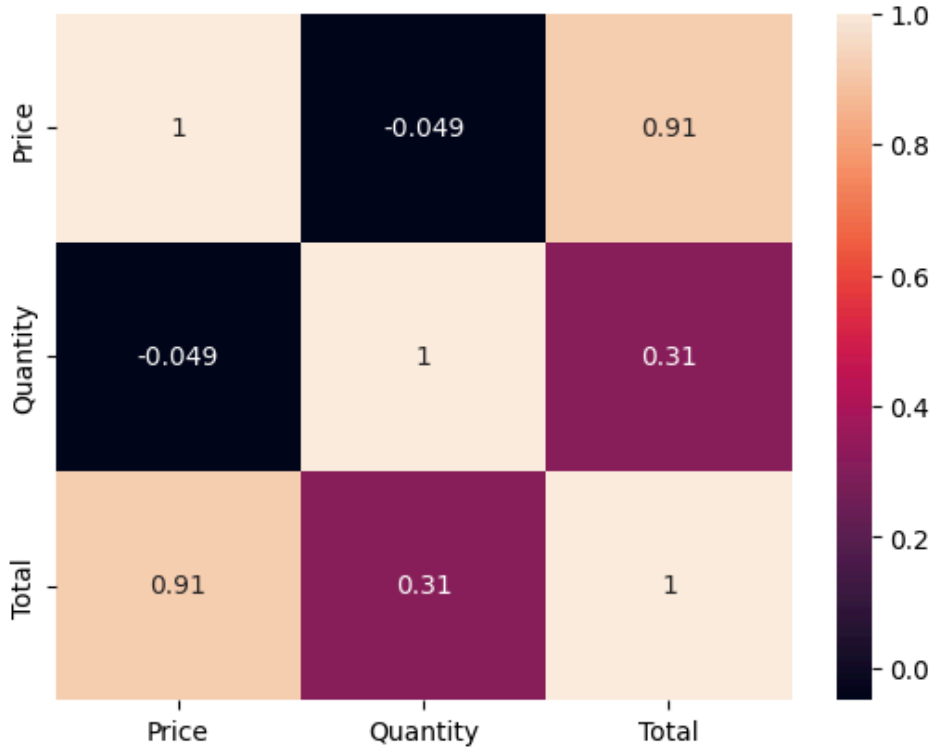
Category
- The Books category has the highest number of orders, indicating strong customer demand in this segment and making it a key revenue driver.

Product
- Product sales are relatively evenly distributed, suggesting that the business does not rely heavily on a single product, which reduces risk.

Heatmap

In [32]:
```
sns.heatmap(df_Clean[Num_column].corr(),annot=True)
```

Out[32]: <Axes: >



Insights

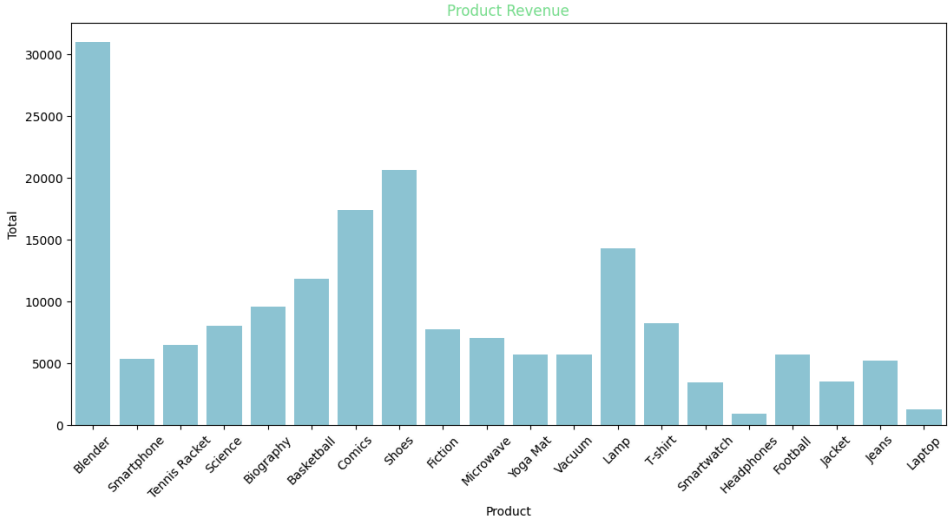
1. Price vs Total → 0.91 (Strong Positive Correlation)
- The correlation between Price and Total is very high (0.91).
- This indicates that as the price increases, the total value of orders also increases significantly.
- This suggests that price is the primary driver of revenue.

2. Quantity vs Total → 0.31 (Moderate Positive Correlation)
- The correlation between Quantity and Total is moderate (0.31).
- This indicates that increasing quantity has some impact on total revenue, but not as strong as price.
- This suggests that bulk purchases contribute to revenue, but are not the main driver.

3. Price vs Quantity → -0.049 (Almost No Correlation)
- The correlation between Price and Quantity is very weak (-0.049).
- This indicates that price does not significantly influence the quantity purchased.
- This suggests that customers' buying quantity is independent of product pricing.

Product Revenue Analysis

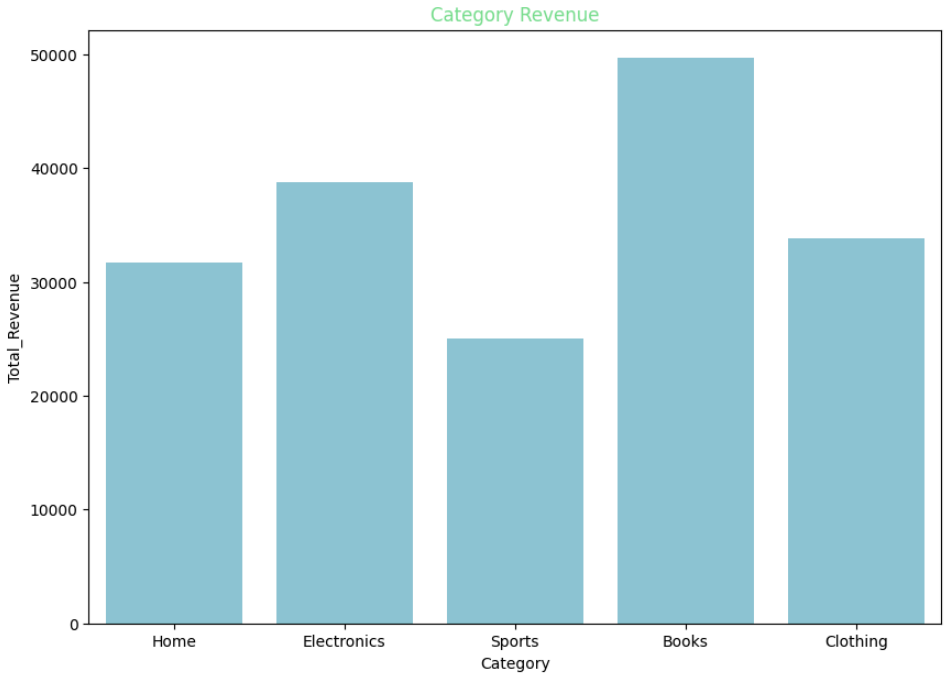
In [33]:
```
Product_Revenue = pd.read_sql("Select Product, sum(Total) as " \
"Total from df_clean group by Product",engine)
plt.figure(figsize=(13,6))
sns.barplot(x=Product_Revenue["Product"], y= Product_Revenue["Total"],
color= "#82ccdd")
plt.title('Product Revenue',fontsize = 12, color = '#78e08f')
plt.xticks(rotation = 45)
plt.show()
print("Insights")
print("--"*70)
print("""The analysis shows that revenue is unevenly distributed across
products,
with a few items like Blender, Shoes, and Comics generating the highest
sales. This indicates that the business relies on a small number of
high-performing products for major revenue. Meanwhile,
some products such as Headphones, Laptop, and Smartwatch contribute very
little,
suggesting lower demand or fewer transactions. Overall, the product
portfolio is diversified,
but improving underperforming products can help increase total
revenue.""")
print('--'*70)
```




```
Insights
------------------------------------------------------------------
------------------------------------------------------------------
--------
The analysis shows that revenue is unevenly distributed across
products,
with a few items like Blender, Shoes, and Comics generating the
highest
sales. This indicates that the business relies on a small number
of
high-performing products for major revenue. Meanwhile,
some products such as Headphones, Laptop, and Smartwatch
contribute very little,
suggesting lower demand or fewer transactions. Overall, the
product portfolio is diversified,
but improving underperforming products can help increase total
revenue.
------------------------------------------------------------------
```

```
----------------------------------------------------------------------
--------
```

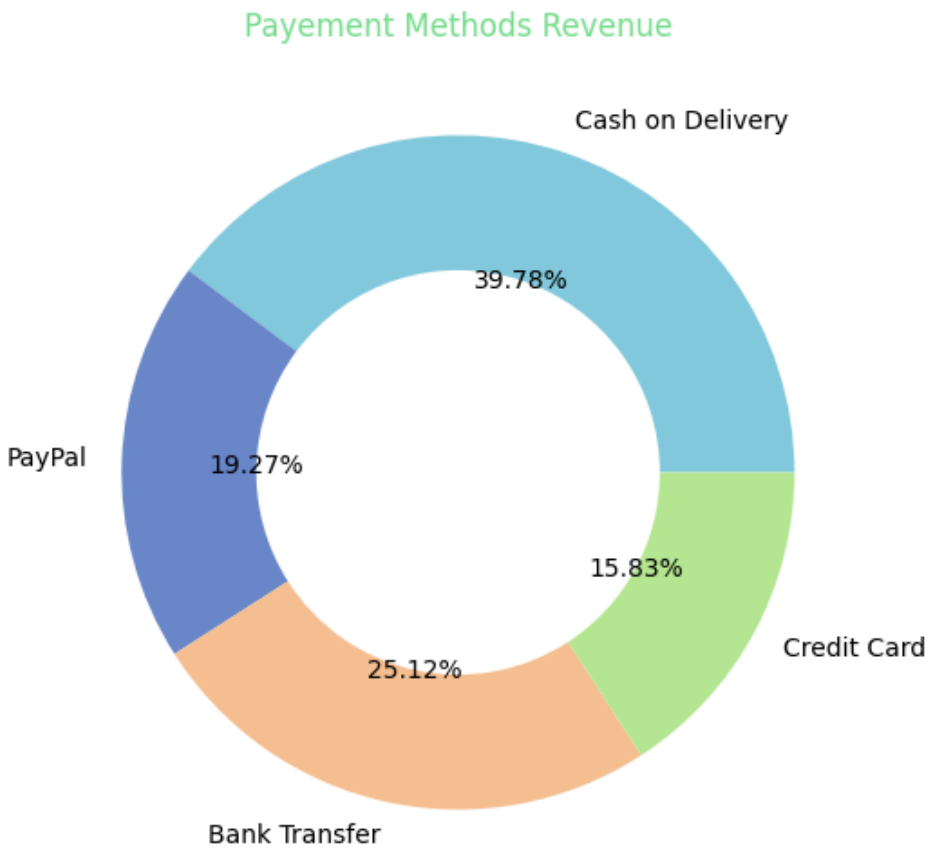
In [34]:
```
Category_Revenue = pd.read_sql('Select Category, sum(Total) as ' \
'Total_Revenue from df_clean group by Category',engine)
plt.figure(figsize=(10,7))
sns.barplot(x="Category", y = "Total_Revenue",
data= Category_Revenue,color='#82ccdd')
plt.title("Category Revenue", fontsize =12,
color = "#78e08f")
plt.show()
print("Insights")
print("--"*70)
print('''The analysis shows that the Books category generates the
highest revenue,
followed by Electronics and Clothing, indicating strong customer demand
in these segments.
In contrast, Sports contributes the least revenue, suggesting lower
demand or fewer sales.
Overall, revenue is uneven across categories, highlighting that focusing
on high-performing
categories like Books can drive business growth, while low-performing
categories may
need improvement strategies.''')
print("--"*70)
```




```
Insights
------------------------------------------------------------------------
------------------------------------------------------------------------
--------
The analysis shows that the Books category generates the highest
revenue,
followed by Electronics and Clothing, indicating strong customer
demand in these segments.
In contrast, Sports contributes the least revenue, suggesting
lower demand or fewer sales.
Overall, revenue is uneven across categories, highlighting that
focusing on high-performing
categories like Books can drive business growth, while low-
```

```
performing categories may
need improvement strategies.
------------------------------------------------------------------
------------------------------------------------------------------
--------
```

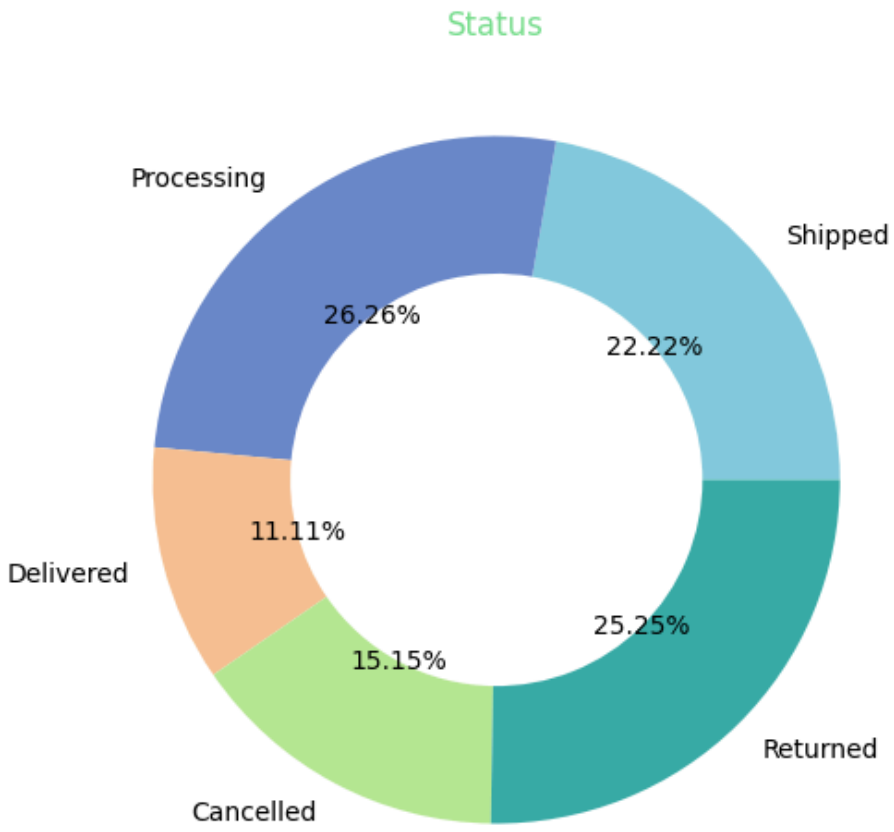
In [35]:
```
Method_Revenue = pd.read_sql("Select Payment_Method, sum(Total) as " \
"Total_Revenue from df_clean group by Payment_Method",engine)
plt.figure(figsize=(6,7))
plt.pie(Method_Revenue['Total_Revenue'], labels=
Method_Revenue['Payment_Method'],
autopct="%1.2f%%",colors=["#82ccdd","#6a89cc","#f8c291","#b8e994"],
wedgeprops={'width':0.4})
plt.title('Payement Methods Revenue', fontsize = 12, color = "#78e08f")
plt.show()
print("Insights")
print('--'*70)
print('''The analysis shows that Cash on Delivery is the most preferred
payment method,
accounting for nearly '40%' of transactions, indicating a strong
customer preference for
cash-based payments. Bank Transfer and PayPal have moderate usage, while
Credit Card is
the least used. This suggests that customers may have lower trust in
online payment methods,
which could limit digital payment adoption.''')
print('--'*70)
```




```
Insights
------------------------------------------------------------------------
------------------------------------------------------------------------
--------
The analysis shows that Cash on Delivery is the most preferred
payment method,
accounting for nearly '40%' of transactions, indicating a strong
customer preference for
```

```
cash-based payments. Bank Transfer and PayPal have moderate usage,
while Credit Card is
the least used. This suggests that customers may have lower trust
in online payment methods,
which could limit digital payment adoption.
-----------------------------------------------------------------
-----------------------------------------------------------------
--------
```

In [36]:
```
Status = pd.read_sql("Select Status, count(Status) as Status_Count from
df_clean group by Status", engine)
plt.figure(figsize=(6,7))
plt.pie(Status['Status_Count'], labels= Status['Status'],
autopct="%1.2f%%",colors=["#82ccdd","#6a89cc","#f8c291"
,"#b8e994","#38ada9"],wedgeprops={'width':0.4})
plt.title('Status', fontsize = 12, color = "#78e08f")
plt.show()
print("Insight")
print('--'*70)
print('''The analysis shows that Processing (26.26%) and Returned
(25.25%)
orders make up the largest share, while Delivered orders are
significantly
lower (11.11%). This indicates potential inefficiencies in order
completion and a
high return rate, which may negatively impact customer
satisfaction and overall business performance.''')
print("--"*70)
```



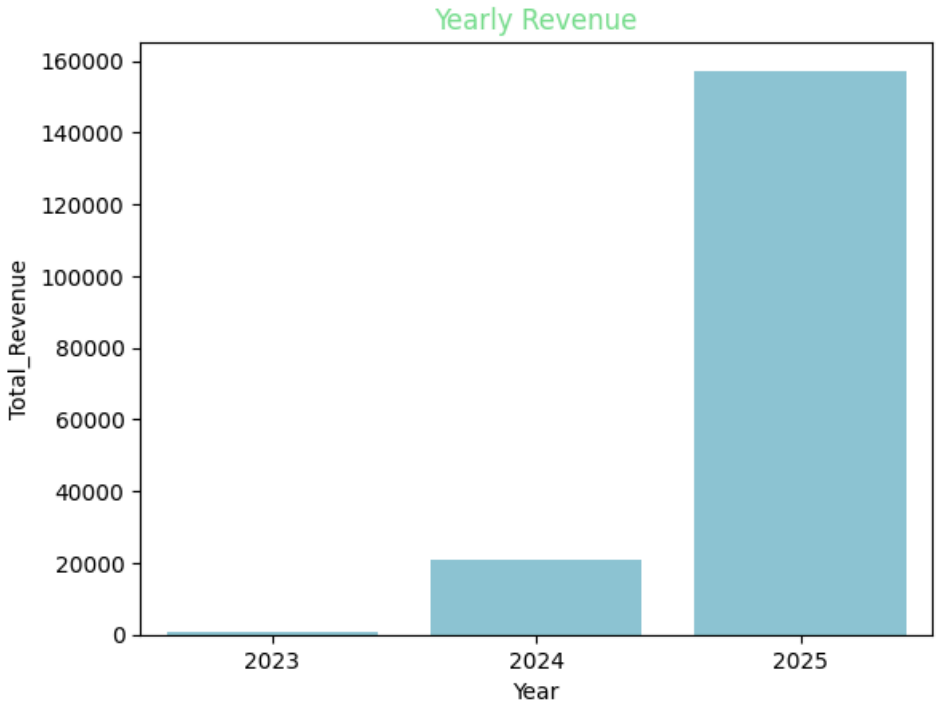

```
Insight
------------------------------------------------------------------------
------------------------------------------------------------------------
--------
The analysis shows that Processing (26.26%) and Returned (25.25%)
orders make up the largest share, while Delivered orders are
significantly
lower (11.11%). This indicates potential inefficiencies in order
completion and a
high return rate, which may negatively impact customer
```

```
satisfaction and overall business performance.
------------------------------------------------------------------
------------------------------------------------------------------
--------
```

Time Based Analysis

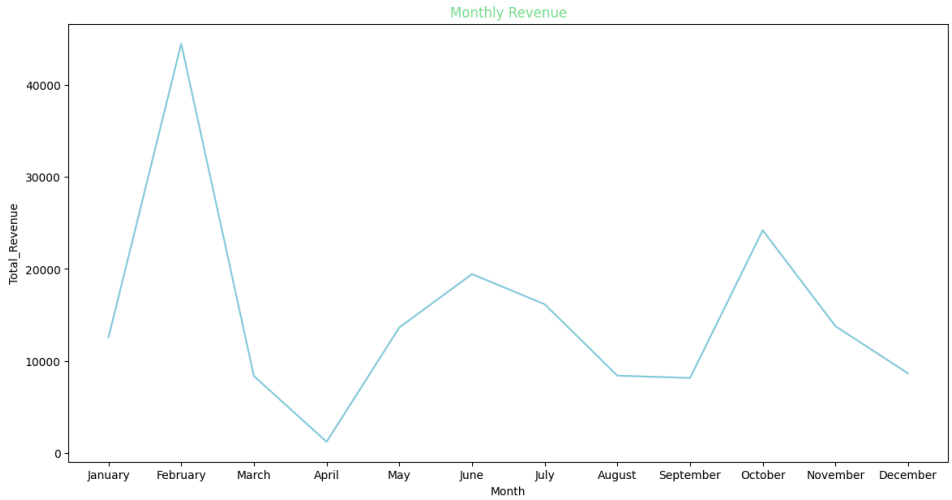
In [53]:

```
Yearly_Revenue = pd.read_sql("Select year(Order_Date) as Year ," \
"sum(Total) as Total_Revenue from df_clean group by Year", engine)
sns.barplot(y = 'Total_Revenue', x = 'Year',
data = Yearly_Revenue,color='#82ccdd')
plt.title("Yearly Revenue", fontsize = 12, color = "#78e08f")
plt.show()
print('Insights')
print('--'*70)
print('''The analysis shows a significant increase in revenue over the
years,
with a sharp rise in 2025 compared to 2023 and 2024. This indicates
strong
business growth and increasing sales performance over time.''')
print("--"*70)
```



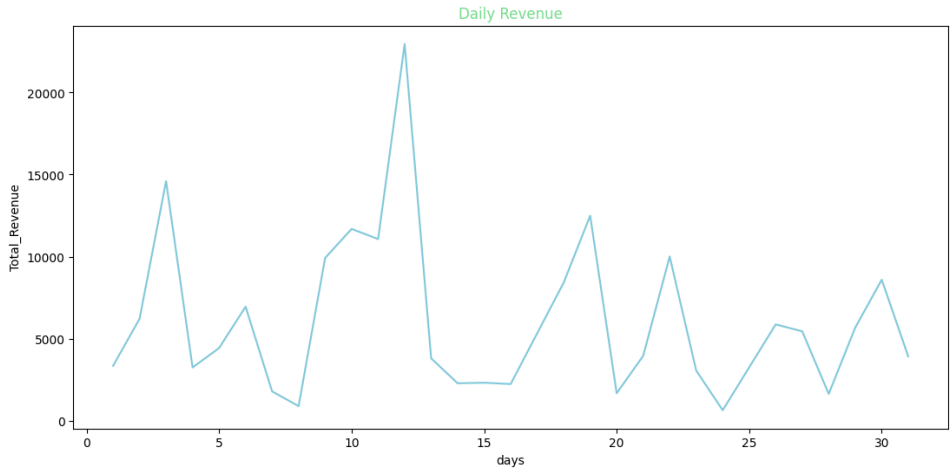

```
Insights
------------------------------------------------------------------
------------------------------------------------------------------
--------
The analysis shows a significant increase in revenue over the
years,
with a sharp rise in 2025 compared to 2023 and 2024. This
indicates strong
business growth and increasing sales performance over time.
------------------------------------------------------------------
------------------------------------------------------------------
--------
```

In [67]:
```
Monthly_Revenue = pd.read_sql("Select MonthName(Order_Date) as Month,
MONTH(Order_Date) AS Month_Num," \
"sum(Total) as Total_Revenue from df_clean group by Month,Month_Num
order by Month_Num",engine)
Monthly_Revenue
plt.figure(figsize=(14,7))
sns.lineplot(x = "Month", y = "Total_Revenue",
data=Monthly_Revenue,color="#82ccdd")
plt.title("Monthly Revenue", fontsize = 12, color = "#78e08f")
plt.show()
print("Insights")
print("--"*70)
print("""The analysis shows that revenue fluctuates significantly across
months,
with a sharp peak in February and another high in October, while April
records
the lowest revenue. This indicates seasonal variations in sales, where
certain months
perform exceptionally well while others experience lower demand.""")
print("--"*70)
```




```
Insights
------------------------------------------------------------------
------------------------------------------------------------------
--------
The analysis shows that revenue fluctuates significantly across
months,
with a sharp peak in February and another high in October, while
April records
the lowest revenue. This indicates seasonal variations in sales,
where certain months
perform exceptionally well while others experience lower demand.
------------------------------------------------------------------
------------------------------------------------------------------
--------
```

In [71]:
```
Daily_Revenue = pd.read_sql("Select day(Order_date) as days, " \
"sum(Total) as Total_Revenue from df_clean group by days order by days",
engine)
plt.figure(figsize=(13,6))
sns.lineplot(x = "days",y="Total_Revenue",
data=Daily_Revenue, color = "#82ccdd")
plt.title("Daily Revenue", fontsize =12, color ="#78e08f")
plt.show()
print('Insights')
print("--"*70)
print("""The daily revenue shows high fluctuations, with a sharp peak
around mid-month and several low-performing days. This indicates
inconsistent
sales performance on a day-to-day basis, where a few days contribute
significantly more revenue than others.""")
print("--"*70)
```




```
Insights
------------------------------------------------------------------------
------------------------------------------------------------------------
--------
The daily revenue shows high fluctuations, with a sharp peak
around mid-month and several low-performing days. This indicates
inconsistent
sales performance on a day-to-day basis, where a few days
contribute
significantly more revenue than others.
------------------------------------------------------------------------
------------------------------------------------------------------------
--------
```

Exported with runcell — convert notebooks to HTML or PDF anytime at runcell.dev.